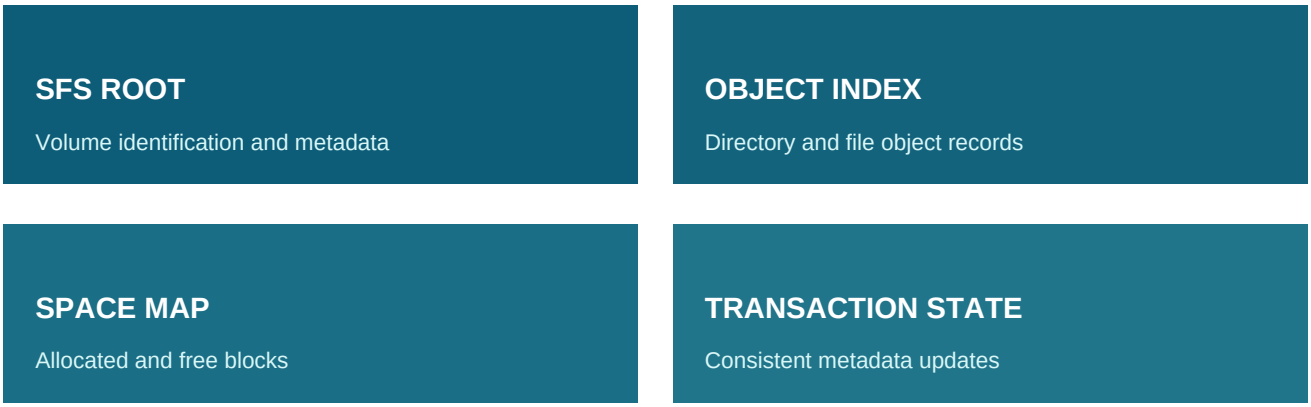
Amiga SmartFileSystem - SFS

OFFICIAL FORMAT / CHRONOLOGY / ENGINEERING

A third-party Amiga filesystem designed for higher performance and improved crash resilience using modernised metadata structures and transactions.

This guide documents the original format's technical architecture, history and independent source references. InfiltratorOS modules and install packages are implementation artefacts, not the authority for the historic on-disk definition.

CONCEPTUAL MEDIA MAP



CONTENTS

1. Historical chronology
2. Original on-disk architecture
3. Allocation and directories
4. Integrity and recovery
5. Design rationale and limitations
6. InfiltratorOS integration boundary

Amiga SmartFileSystem - SFS

HISTORICAL CHRONOLOGY

SmartFileSystem emerged in the late 1990s Amiga ecosystem as an alternative to the original OFS/FFS families. It sought improved directory performance, efficient use of hard disks and stronger resilience to interruption than conventional bitmap updates. The Amiga community and AmigaOS 4 documentation recognise SmartFileSystem as a separate format rather than a variant of the DOS0-DOS5 family.

AmigaOS developer and distribution archives record versions, tools and source components of SmartFileSystem, including object and transaction handling. SFS0 and SFS2 should not be presented as identical disk layouts. The historical source releases and operating-system support tables are important for assessing actual on-media revisions and operational limits.

Amiga SmartFileSystem - SFS

ORIGINAL ON-DISK ARCHITECTURE

SFS uses object-oriented metadata for the file and directory namespace and tracks space allocation independently from the old OFS/FFS layout. Directory access and object lookup are designed for larger collections and more efficient updates. Source archives expose object, transaction, free-space and recovery-related structures; the exact disk fields must be read from the matching source revision.

The persistent root and transaction state must be internally consistent before enabling writes. SFS is not FAT and does not use a FAT12-style next-cluster array for all files; similarly, it does not use the checksummed 24-byte OFS data block header. Object identifiers, free-space structures and transaction ordering are fundamental to understanding its recovery behaviour.

Amiga SmartFileSystem - SFS

ALLOCATION AND DIRECTORIES

File objects describe contents and metadata; directory objects associate names with referenced objects. Efficient directory lookup is a design goal, reducing the overhead that older Amiga filesystem formats encountered in large directories. Free-space tracking supports selecting and reusing storage without needing to traverse all existing file chains.

Names, comments, object references and protection information must be interpreted using the exact SFS variant and its charset policy. Fragmentation management and metadata updates require coordination with transaction state. A disk inspector should never invent field offsets when a historical source tree documents a different revision.

Amiga SmartFileSystem - SFS

INTEGRITY AND RECOVERY

SmartFileSystem designs aim to avoid lengthy validation of a normally interrupted volume by controlling metadata commit state. This is not a promise of full user-data journalling or immunity to corrupted sectors. SFS transaction semantics should be explained from source code and not conflated with ext3 JBD, ZFS transaction groups or NTFS $LogFile.

Recovering an SFS volume requires a consistent root, valid object tree, allocation metadata and acceptable transaction state. One must validate the on-disk format variant and preserve all recoverable versions of metadata before mutation. Format-recognition success does not establish that a damaged filesystem can be safely rewritten.

Amiga SmartFileSystem - SFS

DESIGN RATIONALE AND LIMITATIONS

The major historical step was moving beyond the traditional Amiga root, bitmap and file-header scheme toward metadata structures suitable for larger disks and more frequent changes. That brought improvements in common directory workloads and startup after interruption, but also introduced more complex internal logic than classic Amiga FFS.

Historical SFS implementations have platform and capacity limits, and some older variants have different revision and addressing constraints. SFS can be a valid disk format even if a particular Linux port only supports reading it. Published statements should distinguish the disk standard, vendor claims and a tested driver implementation.

Amiga SmartFileSystem - SFS

INFILTRATOROS INTEGRATION BOUNDARY

SFS.so must independently identify the variant and source-revision format before reporting usable capabilities. Its existence does not qualify on-media write and recovery support. A separate SFS2.so is required when the namespace, addressing or disk structures are incompatible.

The AmigaOS source distribution and filesystem table are the historical reference points. Our application’s .so, its .run installer and a stored kernel adapter do not define the upstream SmartFileSystem format.

Amiga SmartFileSystem - SFS

PRIMARY SOURCE REFERENCES

These primary technical documents and near-contemporary references provide provenance for the design history and on-disk structures. For exact field sizes or a particular format revision, consult the original specifications and source code, not the conceptual cover illustration.

1. AmigaOS file systems comparison, SFS and SFS2

https://wiki.amigaos.net/wiki/UserDoc%3AAmigaOS_File_Systems

2. Original SmartFileSystem source and documentation archive

https://os4depot.net/?file=driver%2Ffilesystem%2Fsfs.lha&function=showcontent

3. Amiga SmartFileSystem background

https://wiki.amiga.org/index.php?title=SmartFileSystem

Our independent .so, optional .ko, build procedures and test evidence describe a separate implementation. The source-level history in this PDF cannot itself qualify production writes, boot, recovery or cross-platform compatibility.

SFS / engineering detail

DISK READ PATH / ENGINEERING NOTES

1. Confirm the SFS0-compatible disk identifier and exact SmartFileSystem generation before interpreting root, object and allocation structures.

2. Read the original volume metadata and determine the current persistent object index. The object namespace is not an OFS/FFS directory-hash layout.

3. Locate file and directory objects through the format-specific object structures; record identity and object linkage must be checked before file-data translation.

4. Reconstruct allocation using free-space and extent metadata instead of assuming a single FAT-style chain of cluster numbers.

5. Interpret modification state and transaction-related metadata together. A completed data write is not proof that its new namespace state was committed.

6. Check object keys, directory entries and the lengths of named objects against the applicable format revision before following pointers.

7. Separate SFS on-disk fields from AmigaDOS handler APIs: a handler may expose familiar filenames while using a wholly different block organisation.

8. Where source revisions disagree on field interpretation, make the stored format revision and original source implementation authoritative.

SFS / engineering detail

VALIDATION, RECOVERY AND COMPATIBILITY

1. Require a coherent root/object mapping and allocation graph before enabling writes; surviving blocks from earlier updates are not automatically current.

2. Cross-check object references against directory membership and ensure that a freed extent is not still reachable from a live object.

3. A metadata update may be interrupted between old and new roots. Preserve both states for analysis rather than assuming the newest-looking block is valid.

4. Verify signatures and record lengths before using object structures. Arbitrary raw block values must not be mistaken for valid object identifiers.

5. Do not load SFS0 media through an SFS2-only decoder or assume either variant is a harmless size extension of the other.

6. Report uncertainty where historical sources do not establish a normative field offset; guessing layout from an illustration risks data corruption.

7. Document what the independent adapter can read and write separately from the original SFS design and source-level capabilities.

8. Treat recovery as a graph-consistency problem, not a generic FAT repair or OFS bitmap rebuild.